

Оглавление

1. Цель работы	2
2. Среда выполнения и реализация стенда	2
3. Реализованная топология	2
4. Настройка IPv4-адресации	2
4.1. Физические интерфейсы	2
4.2. Loopback-интерфейсы	3
5. Статическая маршрутизация	3
5.1. Основные маршруты между loopback-интерфейсами	3
6. Резервный маршрут и отказ линии	4
7. Маршрут по умолчанию	5
8. Итоговая конфигурация и диагностика	6
8.1. Адреса интерфейсов	6
8.2. Финальные таблицы маршрутизации	6
9. Ответы на контрольные вопросы	6
9.1. Когда статический маршрут добавляется в таблицу IP-маршрутизации?	6
9.2. Какой исходный IP будет выбран для ping без явного указания source?	7
10. Вывод	7

1. Цель работы

Цель лабораторной работы — получить практические навыки настройки IPv4-адресов, loopback-интерфейсов и статической маршрутизации, проверить необходимость обратного маршрута, работу резервного статического маршрута при отказе линии и применение маршрута по умолчанию.

Исходная лабораторная работа рассчитана на маршрутизаторы Huawei. В данной реализации функциональность маршрутизаторов воспроизведена средствами Linux в трех Docker-контейнерах без использования FRR. Настройка выполнялась командами `ip addr`, `ip link`, `ip route`, `ping` и `traceroute`.

2. Среда выполнения и реализация стенда

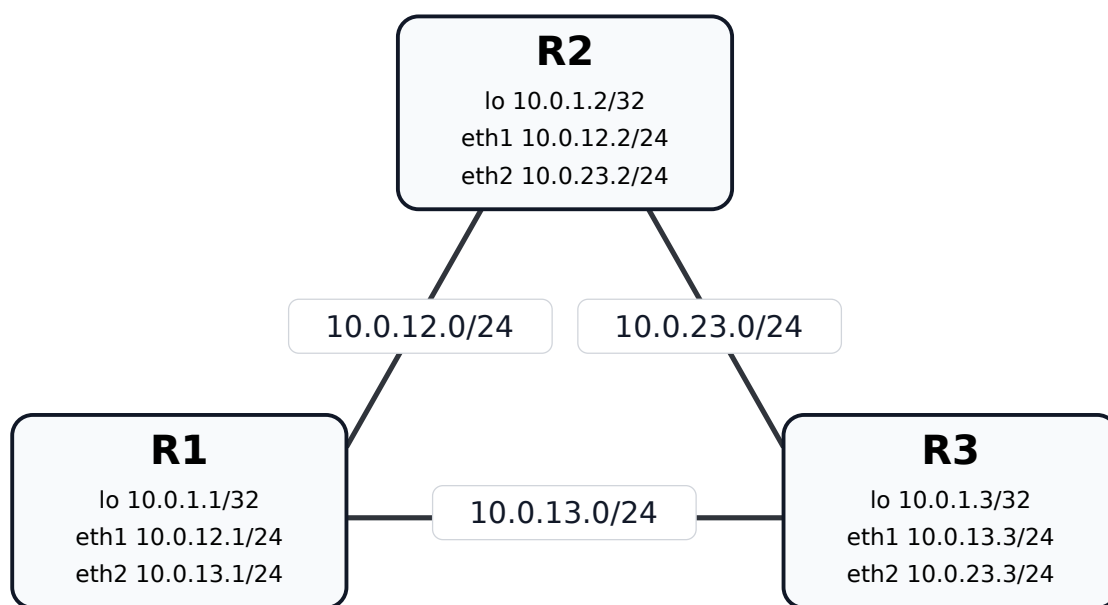
Хостовая система: Arch Linux. Для каждого маршрутизатора создан отдельный Docker-контейнер на базе Ubuntu 24.04. В образ установлены `iproute2`, `iputils-ping`, `traceroute`, `procps` и `tcpdump`. Контейнеры запускались без стандартной Docker-сети (`--network none`), с разрешениями `NET_ADMIN`, `NET_RAW` и включенной пересылкой IPv4-пакетов (`net.ipv4.ip_forward=1`).

Виртуальные линии между маршрутизаторами созданы как пары `veth`. Каждый конец пары перенесен в `network namespace` соответствующего контейнера и переименован в `eth1` или `eth2`.

```
# пример создания одного виртуального линка R1-R2
ip link add r1-r2 type veth peer name r2-r1
ip link set r1-r2 netns <PID_R1>
ip link set r2-r1 netns <PID_R2>
```

Такая реализация позволяет работать с независимыми интерфейсами и таблицами маршрутизации каждого устройства, при этом все контейнеры используют одно ядро Linux хоста.

3. Реализованная топология



Связи реализованы парами `veth`; R1, R2 и R3 — отдельные Docker network namespace.

Figure 1: Реализованная топология и адресация

Использованы три транзитные сети: `10.0.12.0/24`, `10.0.13.0/24`, `10.0.23.0/24`. Для имитации конечных сетей/клиентов на loopback-интерфейсах используются адреса `10.0.1.1/32`, `10.0.1.2/32`, `10.0.1.3/32`.

4. Настройка IPv4-адресации

4.1. Физические интерфейсы

Настройка адресов выполнялась следующими командами:

```
# R1
ip addr add 10.0.12.1/24 dev eth1
ip addr add 10.0.13.1/24 dev eth2

# R2
ip addr add 10.0.12.2/24 dev eth1
ip addr add 10.0.23.2/24 dev eth2

# R3
ip addr add 10.0.13.3/24 dev eth1
ip addr add 10.0.23.3/24 dev eth2
```

После назначения адресов Linux автоматически добавил маршруты к непосредственно подключенным сетям. Например, на R1:

```
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.1
10.0.13.0/24 dev eth2 proto kernel scope link src 10.0.13.1
```

Связность проверялась по одному разу для каждого физического сегмента. Например, для линии R1–R2 непосредственно после настройки адресов:

```
R1# ping -c 2 -I 10.0.12.1 10.0.12.2
64 bytes from 10.0.12.2: icmp_seq=1 ttl=64 time=0.061 ms
64 bytes from 10.0.12.2: icmp_seq=2 ttl=64 time=0.059 ms
2 packets transmitted, 2 received, 0% packet loss
```

Аналогичная проверка двух остальных непосредственно подключенных сетей также дала 0 % потерь. Повторять одинаковый вывод для каждой линии в отчете нецелесообразно: эта проверка подтверждает только адресацию и состояние канального соединения, но еще не работу статических маршрутов к loopback-адресам.

4.2. Loopback-интерфейсы

В Linux использован уже существующий интерфейс `lo`, которому назначены дополнительные адреса:

```
# R1
ip addr add 10.0.1.1/32 dev lo
# R2
ip addr add 10.0.1.2/32 dev lo
# R3
ip addr add 10.0.1.3/32 dev lo
```

Собственный loopback-адрес помещается Linux в локальную таблицу маршрутизации. Например, на R1 диагностическая команда `ip route show table local` показала:

```
local 10.0.1.1 dev lo proto kernel scope host src 10.0.1.1
```

5. Статическая маршрутизация

5.1. Основные маршруты между loopback-интерфейсами

Для достижения loopback-адресов соседних маршрутизаторов были добавлены статические маршруты:

```
# R1
ip route add 10.0.1.2/32 via 10.0.12.2
ip route add 10.0.1.3/32 via 10.0.13.3

# R2
ip route add 10.0.1.1/32 via 10.0.12.1
ip route add 10.0.1.3/32 via 10.0.23.3
```

```
# R3
ip route add 10.0.1.1/32 via 10.0.13.1
ip route add 10.0.1.2/32 via 10.0.23.2
```

Проверка R1–R2 проводилась поэтапно. Сначала маршрут был добавлен только на R1. Команда `ip route get` подтверждает, что R1 уже умеет отправить пакет к R2, однако `ping` не получает ни одного ответа:

```
R1# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.12.2 dev eth1

R1# ping -c 3 -W 1 -I 10.0.1.1 10.0.1.2
3 packets transmitted, 0 received, 100% packet loss

R2# ip route get 10.0.1.1 from 10.0.1.2
RTNETLINK answers: Network is unreachable
```

Причина не в маршруте запроса: echo request доходит до R2, но R2 не знает, куда отправить echo reply с назначением 10.0.1.1. После добавления на R2 обратного маршрута `ip route add 10.0.1.1/32 via 10.0.12.1` повторная проверка сразу стала успешной:

```
R1# ping -c 3 -W 1 -I 10.0.1.1 10.0.1.2
64 bytes from 10.0.1.2: icmp_seq=1 ttl=64 time=0.201 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=64 time=0.049 ms
64 bytes from 10.0.1.2: icmp_seq=3 ttl=64 time=0.046 ms
3 packets transmitted, 3 received, 0% packet loss
```

Таким образом, для двустороннего обмена нужны маршрут к адресу назначения и обратный маршрут к адресу источника. После симметричной настройки пар R1–R3 и R2–R3 loopback-интерфейсы всех трех устройств стали взаимно достижимы.

Проверка	Результат	Назначение
R1 10.0.1.1 → R2 10.0.1.2	0 % потерь	проверка маршрутов R1↔R2
R1 10.0.1.1 → R3 10.0.1.3	0 % потерь	проверка маршрутов R1↔R3
R2 10.0.1.2 → R3 10.0.1.3	0 % потерь	проверка маршрутов R2↔R3

6. Резервный маршрут и отказ линии

Для направления R1↔R2 был создан резервный путь через R3. В исходном оборудовании Huawei для выбора резервного маршрута используется параметр `preference`; в Linux аналогичный выбор среди статических маршрутов был воспроизведен с помощью `metric` — маршрут с меньшим значением выбирается первым.

```
# основной путь
ip route add 10.0.1.2/32 via 10.0.12.2 metric 60 # R1
ip route add 10.0.1.1/32 via 10.0.12.1 metric 60 # R2

# резерв через R3
ip route add 10.0.1.2/32 via 10.0.13.3 metric 100 # R1
ip route add 10.0.1.1/32 via 10.0.23.3 metric 100 # R2
```

До отказа `ip route get` на R1 выбирал прямой next hop, а ответ от соседнего loopback приходил с TTL 64:

```
R1# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.12.2 dev eth1

R1# ping -c 2 -I 10.0.1.1 10.0.1.2
64 bytes from 10.0.1.2: icmp_seq=1 ttl=64 time=0.038 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=64 time=0.032 ms
2 packets transmitted, 2 received, 0% packet loss
```

Для моделирования отказа линия R1-R2 была административно отключена с обеих сторон:

```
ip link set eth1 down # R1
ip link set eth1 down # R2
```

После отключения основной маршрут исчез из активной таблицы, и R1 стал использовать резервный next hop 10.0.13.3. Связь между loopback-интерфейсами сохранилась; TTL ответа уменьшился с 64 до 63, поскольку ответ теперь прошел через дополнительный маршрутизатор R3:

```
R1# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.13.3 dev eth2

R1# ping -c 3 -I 10.0.1.1 10.0.1.2
64 bytes from 10.0.1.2: icmp_seq=1 ttl=63 time=0.085 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=63 time=0.049 ms
64 bytes from 10.0.1.2: icmp_seq=3 ttl=63 time=0.040 ms
3 packets transmitted, 3 received, 0% packet loss
```

Наиболее наглядная диагностическая команда — traceroute:

```
traceroute to 10.0.1.2 (10.0.1.2)
1  10.0.13.3
2  10.0.1.2
```

Таким образом, после отказа прямого сегмента трафик действительно прошел по пути R1 → R3 → R2.

Отдельно отмечено различие состояний интерфейса Linux: после отключения одного конца veth второй конец оставался административно UP, но имел NO-CARRIER, а маршруты через него отображались с признаком linkdown. После административного down второго конца соответствующие статические маршруты были удалены из таблицы. При последующем ip link set eth1 up Linux восстановил подключенный маршрут 10.0.12.0/24, но не удаленный статический маршрут /32: его потребовалось добавить повторно. Это отличие важно учитывать при моделировании отказа административным отключением интерфейса.

7. Маршрут по умолчанию

После проверки резервирования линия R1–R2 была включена снова, а прямой обратный маршрут на R2 восстановлен. Специфические маршруты R1 к 10.0.1.2/32 были удалены. До добавления default route диагностическая команда подтвердила отсутствие подходящего пути:

```
R1# ip route get 10.0.1.2 from 10.0.1.1
RTNETLINK answers: Network is unreachable
```

На R1 был добавлен маршрут по умолчанию:

```
ip route add default via 10.0.12.2
```

Итоговая таблица R1:

```
default via 10.0.12.2 dev eth1
10.0.1.3 via 10.0.13.3 dev eth2
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.1
10.0.13.0/24 dev eth2 proto kernel scope link src 10.0.13.1
```

При этом 10.0.1.2 достигается через маршрут по умолчанию, а для 10.0.1.3 Linux сохраняет более специфичный маршрут /32 через R3. Это демонстрирует правило longest prefix match: наиболее специфичный совпавший префикс имеет приоритет над 0.0.0.0/0.

После настройки default route тот же запрос стал разрешаться через R2:

```

R1# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.12.2 dev eth1

R1# ping -c 2 -I 10.0.1.1 10.0.1.2
64 bytes from 10.0.1.2: icmp_seq=1 ttl=64 time=0.046 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=64 time=0.030 ms
2 packets transmitted, 2 received, 0% packet loss

R1# traceroute -n -s 10.0.1.1 10.0.1.2
1 10.0.1.2 0.153 ms

```

Связь R1→R2 восстановилась, а traceroute снова показал прямой путь в один переход.

8. Итоговая конфигурация и диагностика

8.1. Адреса интерфейсов

Устройство	Интерфейс	IPv4-адрес
R1	lo	10.0.1.1/32
	eth1	10.0.12.1/24
	eth2	10.0.13.1/24
R2	lo	10.0.1.2/32
	eth1	10.0.12.2/24
	eth2	10.0.23.2/24
R3	lo	10.0.1.3/32
	eth1	10.0.13.3/24
	eth2	10.0.23.3/24

8.2. Финальные таблицы маршрутизации

```

R1
default via 10.0.12.2 dev eth1
10.0.1.3 via 10.0.13.3 dev eth2
10.0.12.0/24 dev eth1 ...
10.0.13.0/24 dev eth2 ...

R2
10.0.1.1 via 10.0.12.1 dev eth1
10.0.1.1 via 10.0.23.3 dev eth2 metric 100
10.0.1.3 via 10.0.23.3 dev eth2
10.0.12.0/24 dev eth1 ...
10.0.23.0/24 dev eth2 ...

R3
10.0.1.1 via 10.0.13.1 dev eth1
10.0.1.2 via 10.0.23.2 dev eth2
10.0.13.0/24 dev eth1 ...
10.0.23.0/24 dev eth2 ...

```

Для диагностики в ходе работы использовались команды `ip -br addr`, `ip -br link`, `ip route`, `ip route show table local`, `ip route get`, `ping` и `traceroute`. В отчет включены только результаты, необходимые для подтверждения ключевых этапов, без дублирования однотипного терминального вывода.

9. Ответы на контрольные вопросы

9.1. Когда статический маршрут добавляется в таблицу IP-маршрутизации?

Статический маршрут может быть активным, когда указанный next hop достижим через рабочий интерфейс/ существующий маршрут. Если следующий переход недоступен, такой маршрут не может использоваться для

пересылки пакетов. В выполненном стенде это наблюдалось при отказе линии: после административного отключения интерфейса маршруты через него переставали участвовать в выборе пути, и использовался резервный маршрут.

9.2. Какой исходный IP будет выбран для ping без явного указания source?

Маршрутизатор выбирает исходный адрес в соответствии с выбранным выходным интерфейсом. Для прямого маршрута R1→R2 через сеть 10.0.12.0/24 источником стал бы адрес 10.0.12.1, а не loopback 10.0.1.1. Поэтому для проверки именно связи loopback-to-loopback источник указывался явно (ping -I 10.0.1.1 ...; в Huawei — параметр -a).

10. Вывод

В ходе лабораторной работы была создана трехмаршрутизаторная IP-сеть в Docker-контейнерах без специализированного routing-daemon. Настроены физические и loopback-адреса, статические маршруты, резервный маршрут и маршрут по умолчанию. Экспериментально подтверждены необходимость обратного маршрута, автоматическое переключение на резервный путь при отказе линии и правило выбора наиболее специфичного маршрута. Использование Linux network namespace и veth позволило воспроизвести основные сетевые процессы исходной Huawei-лабораторной стандартными средствами ядра Linux.